RELATÓRIO DO PROJETO

Markos: Compilador de Markdown Simplificado

Hugo Oliveira · Lígia Martins
UC 00618 · Criar aplicações em linguagem de programação Python

2026

Índice

1. Introdução e Objetivos 3
Objetivos Específicos 3
2. Desenho e Estrutura 4
2.1 Arquitetura do Software 4
2.2 Fluxo de Processamento 5
2.3 Diagrama de Classes (simplificado) 5
2.4 Diagrama de Estado 6
3. Implementação 8
3.1 Entrada e Saída 8
3.2 Processamento de Markdown 8
Títulos e Cabeçalhos 8
Parágrafos 9
Listas 9
Elementos Inline 9
3.3 Backend HTML 10
3.4 Funcionalidades Extra Implementadas 10
3.5 Ambiente e Dependências 10
4. Testes 11
4.1 Estratégia de Teste 11
4.2 Casos Limite Identificados 11
5. Conclusão 12
Dificuldades e Soluções 12
Lições Aprendidas 13
Apêndice — Diagramas de Estado 14
Imagem 1 — Diagrama de ciclo de vida das classes principais 14
Imagem 2 — Máquina de estados para identificar e contar palavras 14

1. Introdução e Objetivos

O presente projeto consiste no desenvolvimento de um compilador de Markdown simplificado, denominado Markos, cujo objetivo é converter documentos escritos em Markdown para HTML válido e estruturado. Markdown é uma linguagem de marcação leve, de sintaxe minimalista, amplamente utilizada em documentação técnica, plataformas de colaboração como o GitHub e geradores de sites estáticos.

A principal motivação para a criação de compiladores deste tipo reside na necessidade de separar o conteúdo da sua apresentação: o autor escreve em texto simples e legível, enquanto o compilador se encarrega de produzir o markup HTML necessário para a renderização em browsers ou outras ferramentas.

Este relatório documenta o trabalho de conclusão da UC 00618, desenvolvido em dupla com base em mais de 18 horas de gravações disponibilizadas pelo formador. A abordagem colaborativa permitiu revisão mútua de código, divisão de tarefas e discussão contínua sobre as decisões de implementação.

Objetivos Específicos

- Implementar a identificação e conversão de títulos e cabeçalhos de níveis 1 a 3 (!Título!, #, ##, ###).
- Processar parágrafos e blocos de texto contínuos delimitados por linhas em branco.
- Gerar listas não-numeradas (-) e listas numeradas (1., 1)), com suporte a parágrafos internos e continuidade de linhas.
- Reconhecer elementos inline de negrito (**palavra**) e itálico (*palavra*).
- Gerir entrada e saída de ficheiros Markdown e HTML, incluindo stdin/stdout, folhas de estilo externas (--style-sheet) e saída formatada (--pretty).

Contexto técnico
O Markos foi desenvolvido inteiramente em Python, fazendo uso extensivo de expressões regulares (módulo re), máquinas de estado finitas para controlo de fluxo e padrões de design modular inspirados no padrão Backend/Frontend. Esta abordagem visa facilitar a extensão futura para novos elementos Markdown ou backends de saída alternativos (e.g. LaTeX, PDF).

2. Desenho e Estrutura

2.1 Arquitetura do Software

O Markos segue uma arquitetura em camadas que separa claramente a análise do documento Markdown (frontend), a lógica de compilação (compilador) e a geração de HTML (backend). Esta separação facilita a substituição de qualquer uma das camadas sem impacto nas restantes.

O fluxo geral de execução é o seguinte: o ficheiro Markdown é lido linha a linha; cada linha é classificada por tipo (título, cabeçalho, lista, parágrafo, linha em branco); a máquina de estados transita entre modos de processamento conforme o tipo detetado; e o backend HTML emite as tags correspondentes.

Módulo	Responsabilidade
`markos.py`	Ficheiro principal: interface de linha de comando, interpretação de argumentos e invocação do compilador.
`markdown_compiler0.py`	Primeira versão do compilador, com suporte inicial a títulos, cabeçalhos e parágrafos.
`markdown_compiler1.py`	Versão avançada: listas, elementos inline e múltiplos níveis de cabeçalhos.
`markdown_backend.py`	Backend abstrato para geração de HTML — métodos de abertura/fecho de elementos.
`markdown_list.py`	Classes auxiliares para listas: MarkdownList, ListItem, ListItemBlock, ListItemHeading.
`html_backend.py`	Backend HTML: gera tags <p>, <h1>–<h6>, <ul>/<ol>, <li>, links, imagens e estilos.
`utils.py`	Funções utilitárias: abertura de ficheiros, rewind_one_line, count_consec, regex helpers.
`tests/`	Ficheiros de teste (Test0.md a Test4.md) para validação do compilador.

2.2 Fluxo de Processamento

O compilador processa o documento de entrada de forma sequencial, linha a linha. O diagrama seguinte ilustra as transições de estado principais:

- **Estado IDLE:** aguarda o início de um novo elemento. Uma linha não-vazia desencadeia a transição para o estado adequado.
- **Estado PARAGRAPH:** acumula linhas de texto contínuos. Uma linha em branco encerra o parágrafo e emite … .
- **Estado LIST:** processa itens de lista, incluindo parágrafos internos e múltiplas linhas por item. A saída do estado emite </ul> ou </ol>.
- **Estado HEADING:** detetado por regex no início da linha (#, ##, ###). Emite imediatamente <h1>…</h3> e regressa a IDLE.

Os elementos inline (negrito e itálico) são processados numa passagem separada sobre o conteúdo de cada bloco, após a deteção do tipo de elemento, através de substituições por regex sobre a string acumulada.

2.3 Diagrama de Classes (simplificado)

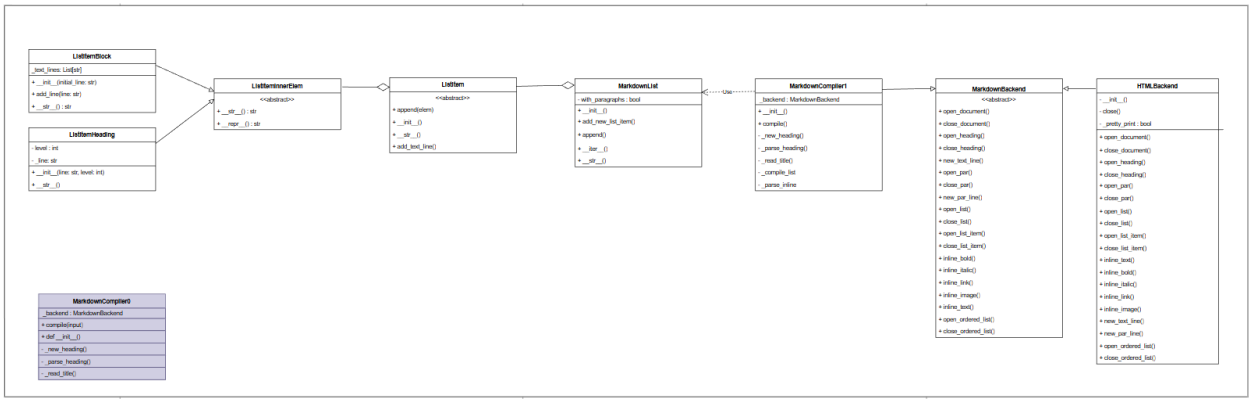

Figura 1 — Diagrama de classes simplificado

2.4 Diagrama de Estado

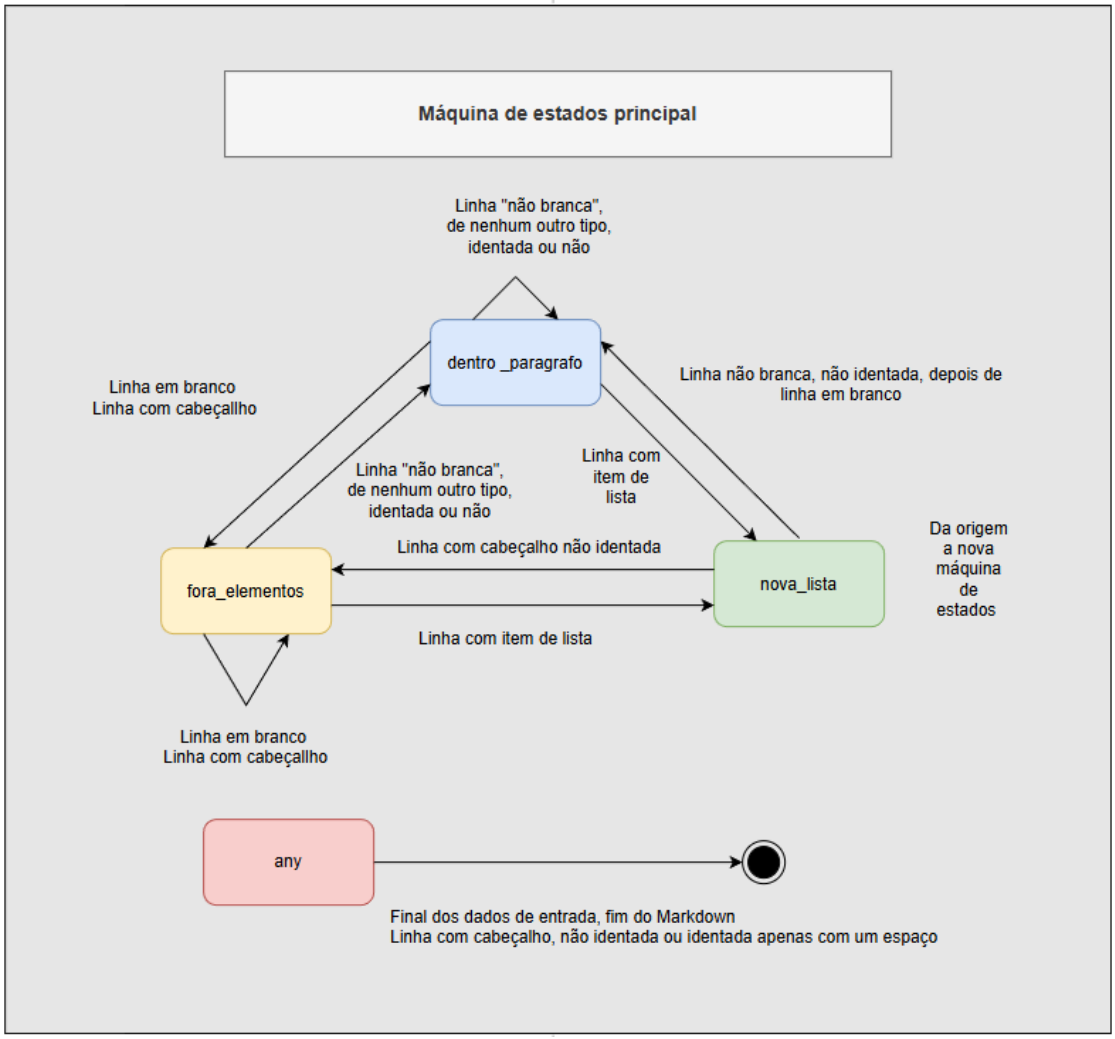

Figura 2 - Máquina de estados principal do compilador

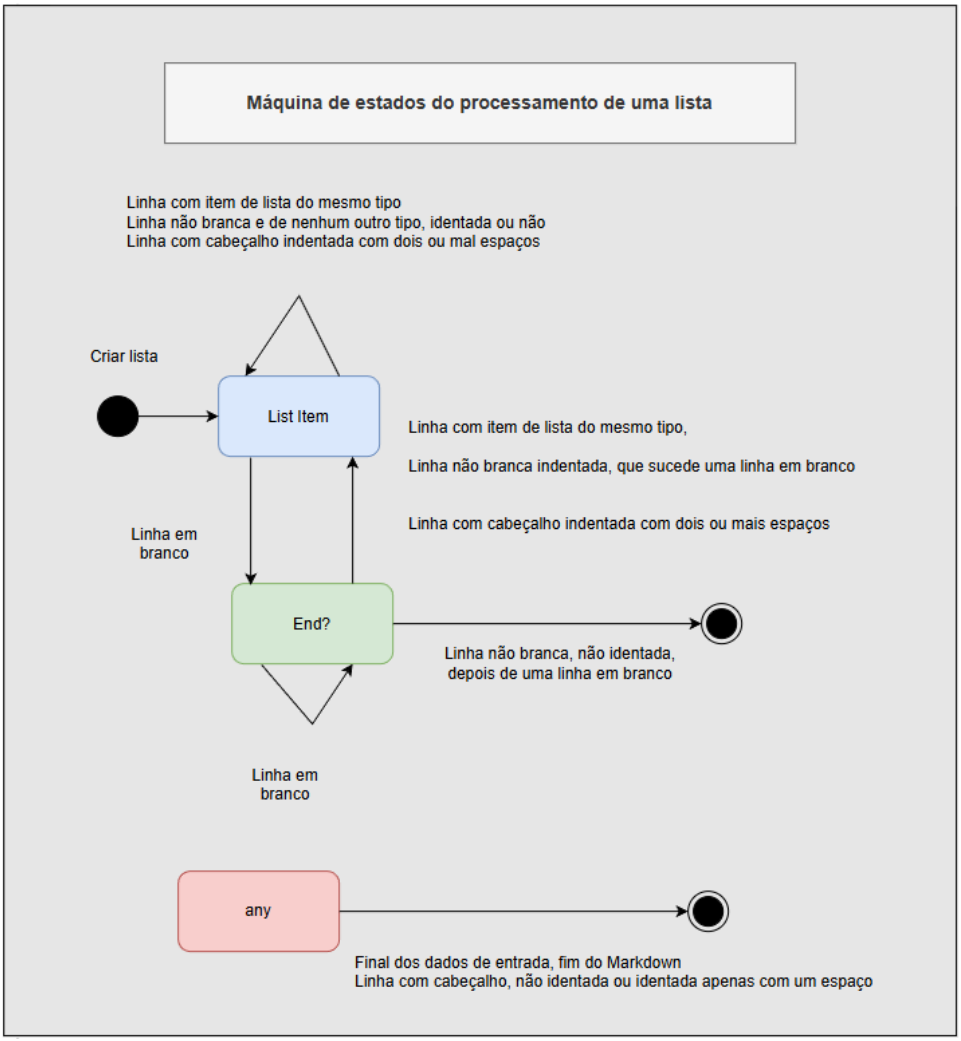

Figura 3 - Máquina de estados para processamento de lista

3. Implementação

3.1 Entrada e Saída

A gestão de entrada e saída foi desenhada para ser flexível, suportando tanto ficheiros em disco como os fluxos standard Unix (stdin/stdout). Isto permite integrar o Markos em pipelines de linha de comando, por exemplo:

```
# Converter ficheiro e guardar resultado
python markos.py input.md -o output.html
# Usar stdin/stdout com pretty-print e folha de estilo
cat input.md | python markos.py --pretty --style-sheet style.css > output.html
```

- Os ficheiros Markdown são lidos do disco ou de stdin com from_file_or_stdin().
- A saída HTML é escrita em ficheiro ou stdout com to_file_or_stdout().
- Parâmetros de linha de comando geridos com docopt:
  - -p ou --pretty → formata a saída HTML com indentação legível usando BeautifulSoup.
  - --style-sheet <css> → injeta <link rel="stylesheet" href="..."> no <head> do HTML gerado.

3.2 Processamento de Markdown

O processamento de cada elemento Markdown segue uma lógica de reconhecimento por padrão, onde cada tipo de linha é identificado por uma expressão regular ou por contexto (estado atual da máquina de estados).

Títulos e Cabeçalhos

O título principal do documento é delimitado por !Título! na primeira linha, seguido obrigatoriamente por uma linha em branco. Os cabeçalhos de secção usam o prefixo # (com espaço obrigatório após o símbolo):

Markdown (input)	HTML gerado (output)

`!Meu Documento!`
 	
`# Secção 1`
`## Subsecção 1.1`
`### Detalhe`
	`<h1>Meu Documento</h1>`
	
	`<h1>Secção 1</h1>`
	`<h2>Subsecção 1.1</h2>`
	`<h3>Detalhe</h3>`

Parágrafos

Blocos de texto contínuo, separados por linhas em branco ou por elementos de bloco (cabeçalhos, listas), são emitidos como parágrafos HTML. Linhas consecutivas sem linha em branco entre elas são concatenadas num único <p>.

Markdown (input)	HTML gerado (output)

`Esta é a primeira linha`
`e continua aqui.`
 	
`Novo parágrafo.`
	`<p>Esta é a primeira linha`
	`e continua aqui.</p>`
	
	`<p>Novo parágrafo.</p>`

Listas

As listas não-numeradas aceitam os marcadores - e * (funcionalidade extra). As listas numeradas reconhecem os padrões 1. e 1). Ambos os tipos suportam parágrafos internos (linhas em branco dentro de um item) e múltiplas linhas por item.

Markdown (input)	HTML gerado (output)

`- Primeiro item`
`- Segundo item`
 	
`1. Item numerado`
`2. Segundo numerado`
	`<ul>`
	`  <li>Primeiro item</li>`
	`  <li>Segundo item</li>`
	`</ul>`
	`<ol>`
	`  <li>Item numerado</li>`
	`  <li>Segundo numerado</li>`
	`</ol>`

Elementos Inline

Os elementos inline são processados após a deteção do bloco contentor, através de substituições por expressão regular sobre a string acumulada. Isto permite que negrito e itálico apareçam dentro de parágrafos, cabeçalhos e itens de lista.

Markdown (input)	HTML gerado (output)

`Texto com **negrito** e`
`*itálico* na mesma linha.`
 	
`__também negrito__`
`_também itálico_`
	`<p>Texto com <strong>negrito</strong>`
	`e <em>itálico</em> na mesma linha.</p>`
	
	`<p><strong>também negrito</strong></p>`
	`<p><em>também itálico</em></p>`

3.3 Backend HTML

O backend HTML é implementado como uma classe que expõe métodos de alto nível para abertura e fecho de elementos, ocultando completamente os detalhes das tags HTML ao compilador. Esta abstração permite, em teoria, substituir o backend por um equivalente LaTeX ou outro formato sem alterar a lógica do compilador.

- <p> para parágrafos; <h1> a <h6> para cabeçalhos.
- <ul> e <ol> para listas; <li> para itens, com suporte a múltiplos <p> internos.
- Elementos inline processados por regex: **…** / __…__ → , *…* / _…_ → .
- Ligações [texto](url) → <a href="url">texto</a> e imagens ![alt](src) → <img> (funcionalidades extra).
- Pretty-print opcional com BeautifulSoup, garantindo HTML indentado e legível para inspeção.

3.4 Funcionalidades Extra Implementadas

Para além do enunciado base, foram implementadas as seguintes extensões, que ampliam a compatibilidade do Markos com a especificação CommonMark:

- Marcador * (asterisco) como alternativa a - em listas não-ordenadas.
- Listas numeradas com as notações 1. e 1), garantindo que ambas produzem <ol>/<li> corretos.
- Negrito e itálico em elementos inline, incluindo dentro de cabeçalhos e itens de lista.
- Ligações inline [texto](url) → <a href="url">texto</a>.
- Imagens inline ![texto alternativo](src) → <img src="src" alt="texto alternativo">.

> **Nota sobre compatibilidade**
> As funcionalidades extra foram implementadas de forma a não quebrar os casos base do enunciado. Todos os ficheiros de teste originais (Test0.md a Test4.md) continuam a produzir saída correta após a adição das extensões.

3.5 Ambiente e Dependências

O projeto foi desenvolvido e testado em Windows 11, com Python 3.14. As bibliotecas externas utilizadas foram:

- **beautifulsoup4** - para formatação (pretty-print) do HTML gerado
- **typing_extensions** - para o decorator `@override`
- **docopt** - para parsing dos argumentos de linha de comando

4. Testes

4.1 Estratégia de Teste

A validação do compilador foi realizada através de um conjunto de cinco ficheiros de teste, cada um cobrindo um subconjunto específico das funcionalidades. A abordagem seguida foi incremental: cada teste adiciona complexidade ao anterior, permitindo isolar regressões quando uma nova funcionalidade é introduzida.

Para cada ficheiro .md existe um ficheiro .html esperado, gerado manualmente como referência. A comparação é feita por diff de texto, ignorando diferenças de espaçamento quando a opção --pretty é usada, e de forma exata quando não é.

Ficheiro	Âmbito	O que é testado
`Test0.md`	**Caso base**	Título principal (!Título!) e parágrafo simples. Valida o arranque do compilador e a geração do bloco <html>/<head>/<body>.
`Test1.md`	**Cabeçalhos**	Cabeçalhos #, ## e ### com e sem espaço obrigatório. Testa múltiplos níveis e a correta emissão de <h1>, <h2>, <h3>.
`Test2.md`	**Elementos inline**	Negrito (**texto** e __texto__) e itálico (*texto* e _texto_) dentro de parágrafos e cabeçalhos. Verifica e .
`Test3.md`	**Listas complexas**	Listas não-numeradas e numeradas com parágrafos internos, múltiplas linhas por item e itens consecutivos. Caso mais exigente.
`Test4.md`	**Funcionalidades extra**	Ligações ([texto](url)), imagens (![alt](src)) e marcador * em listas. Cobre as extensões implementadas além do enunciado base.

4.2 Casos Limite Identificados

Durante os testes, foram identificados vários casos limite que exigiram ajustes no compilador:

1. **Listas com parágrafos internos (Test3.md):** Itens de lista que contêm parágrafos internos (linha em branco dentro do item) exigem que o estado LIST distinga entre fim de item e fim de lista. O critério de saída é a deteção de uma linha não-vazia que não seja prefixo de lista após uma linha em branco.

2. **Cabeçalhos sem espaço obrigatório:** A especificação exige espaço após # ("# Título" é válido; "#Título" não é). A regex foi ajustada para rejeitar o segundo caso e tratá-lo como parágrafo normal.

3. **Inline em contextos aninhados:** Negrito e itálico dentro de itens de lista com parágrafos internos exigiram que o processamento inline fosse aplicado individualmente a cada bloco de texto acumulado, e não à string completa do item.

4. **Fim de ficheiro sem linha em branco final:** O compilador foi ajustado para emitir o bloco corrente (parágrafo ou lista) quando é atingido o fim do ficheiro, mesmo que não exista linha em branco terminadora.

5. Conclusão

O projeto Markos atingiu os objetivos principais, implementando um compilador de Markdown simplificado capaz de gerar HTML válido e estruturado a partir de documentos de texto simples. As funcionalidades nucleares — títulos, cabeçalhos, parágrafos, listas e elementos inline — foram implementadas e validadas com sucesso.

Para além do enunciado base, foram entregues extensões que aumentam a compatibilidade com a especificação CommonMark: suporte ao marcador *, listas numeradas com dupla notação, ligações e imagens inline. Todas as extensões foram integradas sem quebrar os testes originais.

Dificuldades e Soluções

1. **Adaptação do código pré-existente:** O projeto partiu de versões base fornecidas (markdown_compiler0.py e markdown_compiler1.py). Compreender profundamente a arquitetura antes de qualquer modificação foi essencial para evitar regressões. A criação de testes de regressão antes de cada alteração revelou-se uma prática indispensável.
2. **Debugging da máquina de estados:** Os erros mais difíceis de diagnosticar ocorreram em transições de estado incorretas, especialmente em listas com conteúdo complexo. A solução passou por adicionar logging detalhado do estado atual e da linha processada, permitindo reproduzir e isolar cada falha.
3. **Integração do Pretty Print:** O BeautifulSoup reorganiza a árvore HTML ao formatar, o que inicialmente causava alterações na hierarquia de elementos gerados. O problema foi resolvido garantindo que o HTML emitido pelo backend era sempre sintaticamente válido antes de ser passado ao formatador.
4. **Coordenação em dupla:** A divisão de tarefas exigiu definição clara de interfaces entre módulos antes do desenvolvimento. O uso de comentários nas funções públicas de cada módulo e do recurso “live share” reduziu significativamente os problemas de integração.

Lições Aprendidas

- Compreender código pré-existente exige tanta atenção e rigor quanto escrever do zero — em alguns casos, mais.
- Testes incrementais e de regressão são indispensáveis em projetos com múltiplas funcionalidades interdependentes.
- A modularidade por camadas (compilador / backend) facilitou a adição de funcionalidades extra sem impacto no núcleo do sistema.
- Expressar as invariantes do sistema (e.g. "o backend só é chamado a partir de IDLE") em comentários reduz erros de integração em trabalho colaborativo.

Em resumo, o projeto consolidou competências em Python avançado, expressões regulares, máquinas de estado, design modular e metodologias de teste, cumprindo plenamente os objetivos da unidade curricular e produzindo uma base sólida para extensões futuras.

Apêndice — Diagramas de Estado

Imagem 1 — Diagrama de ciclo de vida das classes principais
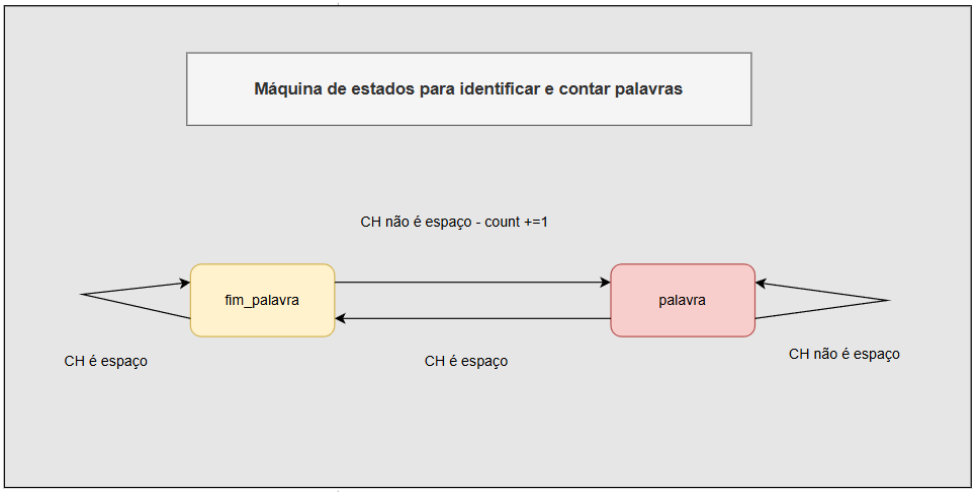
Imagem 2 — Máquina de estados para identificar e contar palavras